FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 4

Name: Aufwand in h:

Mat.Nr: Punkte:

Übungsgruppe: korrigiert:

Beispiel 1 (24 Punkte) Bilderverwaltung: Erstellen Sie eine Software zur Erzeugung und Verwaltung von Bildern mit graphischen Objekten. Die Bilder sind in einer Bilderverwaltung abgelegt und können durch generieren einer SVG-Datei in einem entsprechenden Browser angezeigt werden.

Eine Methode `WriteSVG` soll das Bild als SVG-Datei (*scalable vector graphics*) mit dem gegebenen Namen abspeichern. Der Name der SVG-Datei wird bei der Konstruktion eines Bildes festgelegt. Ein Beispiel für eine solche SVG-Datei könnte z.B. folgendermaßen aussehen:

```
<?xml version="1.0"?>
<svg xmlns="http://www.w3.org/2000/svg">
<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
</svg>
```

Die SVG-Datei beschreibt die Bestandteile der Grafik (im obigen Beispiel ein Rechteck und ein Kreis). Stroke (`stroke="black"`) gibt die Farbe an und `fill` die Füllung. Wird `fill="none"` weggelassen, so wird das Objekt mit der angegebenen Farbe gefüllt.

Es sollen gefüllte und nicht gefüllte Kreise und Rechtecke erzeugt werden, wobei ein Bild nur gefüllte oder nur nicht gefüllte Objekte haben kann.

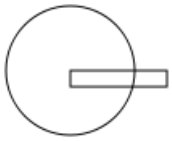


Bild1:test1.svg



Bild2: test2.svg

Die Bilderverwaltung und die Fabriken zur Objekterzeugung exisitieren nur einmal und werden beim

Beenden des Programmes automatisch freigegeben. Verwenden Sie zur Implementierung eine vorgestellte Methode aus der Vorlesung. Graphische Objekte werden mit einer entsprechenden Fabrik erzeugt und das Kopieren und Zuweisen soll verhindert werden!

Die Bilderverwaltung kann mit verschiedenen Fabriken arbeiten und so verschiedene Bilder erzeugen. Die Fabriken in der Bilderverwaltung sollen austauschbar sein!

Das Design soll so ausgelegt sein, dass in Zukunft nicht nur gefüllte und nicht gefüllte graphische Objekte erzeugt werden können, sondern auch z.B. schraffierte oder gepunktete graphische Objekte.

Für die SVG-Datei muss keine Grammatik angegeben werden!

Entwerfen Sie ein Klassendiagramm und verwenden Sie dazu entsprechende Design Pattern. Implementieren Sie das vollständige Klassendiagramm und schreiben sie einen Testtreiber. Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)! Geben Sie die vom Testtreiber erzeugten SVG-Dateien sowohl textuell als auch graphisch ab.

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

HSD
OBERÖSTERREICH
FH-HAGENBERG

Systemdokumentation

Player-Schnittstelle

Version 1.0

J. Rieger, L. Kieweg

Hagenberg, 28. Oktober 2026

Dieses Dokument zeigt den prinzipiellen Aufbau einer Systemdokumentation für Software-Projekte. Die einzelnen Kapitel sind mit Kommentaren versehen, welche die Struktur und den Inhalt erläutern.

Inhaltsverzeichnis

1 Organisatorisches **2**
1.1 Team . . . 2
1.2 Aufteilung der Verantwortungsbereiche . . . 2
1.3 Aufwand . . . 2

2 Anforderungsdefinition (Systemspezifikation) **3**

3 Systementwurf **4**
3.1 Klassendiagramm . . . 4
3.2 Designentscheidungen . . . 5

4 Komponentenentwurf **5**

5 Testprotokollierung **6**
5.1 Ausgabe Konsole Testtreiber . . . 6

6 Quellcode **8**

1 Organisatorisches

1.1 Team

- Julian Rieger, Matr.-Nr: S2510306023, E-Mail: S2510306023@fhooe.at
- Leander Kieweg, Matr.-Nr: S2510306014, E-Mail: S2510306014@fhooe.at

1.2 Aufteilung der Verantwortungsbereiche

- Julian Rieger
  - Implementierung der Klassen: Video, PlayerClient, VideoPlayer, VideoAdapter
  - Implementierung des Testtreibers
  - Verfassen der Systemdokumentation
- Leander Kieweg
  - Implementierung der Klassen: Player, MusicAdapter, MusicPlayer, Media, Song
  - Design des Klassendiagrams
  - Implementierung des Testtreibers

1.3 Aufwand

- Julian Rieger: geschätzt: 7h, tatsächlich 7h
- Leander Kieweg: geschätzt 7h, tatsächlich 7h

2 Anforderungsdefinition (Systemspezifikation)

Es wird eine Software zur generischen Bedienung von zwei verschiedenen Schnittstellen, die eines Video Players und die eines Music Players, und stellt eine einzige, für beide verwendbare Schnittstelle, bereit. Die Schnittstelle soll sieben verschiedene Methoden zur Verfügung stellen, die die Geräte steuern können.

Funktionale Anforderungen

- **Play():** Startet die Wiedergabe des Titels.
- **Stop():** Beendet die Wiedergabe des Titels.
- **Next():** Springt in der Playlist auf den nächsten Titel.
- **Prev():** Springt in der Playlist auf den vorherige Titel.
- **VolInc():** Erhöht die Lautstärke um den Wert 1.
- **VolDec():** Verringert die Lautstärke um den Wert 1.
- **Select(const std::string name):** Wählt einen bestimmten, in der Playlist vorhandenen, Titel aus und startet die Wiedergabe.

Fehlerbehandlung

Fehler bei der Anwendung oder Bedienung des Programms werden abgefangen und es werden Rückmeldungen per std::cerr auf die Konsole ausgegeben und die Funktion wird mit return umgehend wieder verlassen. Zu diese Art von Fehlern zählen jegliche Fehlbedienungen der Schnittstelle. (z.B. Aufruf von Play bei leerer Playlist) Dabei kommt es zu keinen Einschränkungen der darauffolgenden Funktionsaufrufen, sofern ein korrekter Aufruf stattfindet.

3 Systementwurf

3.1 Klassendiagramm

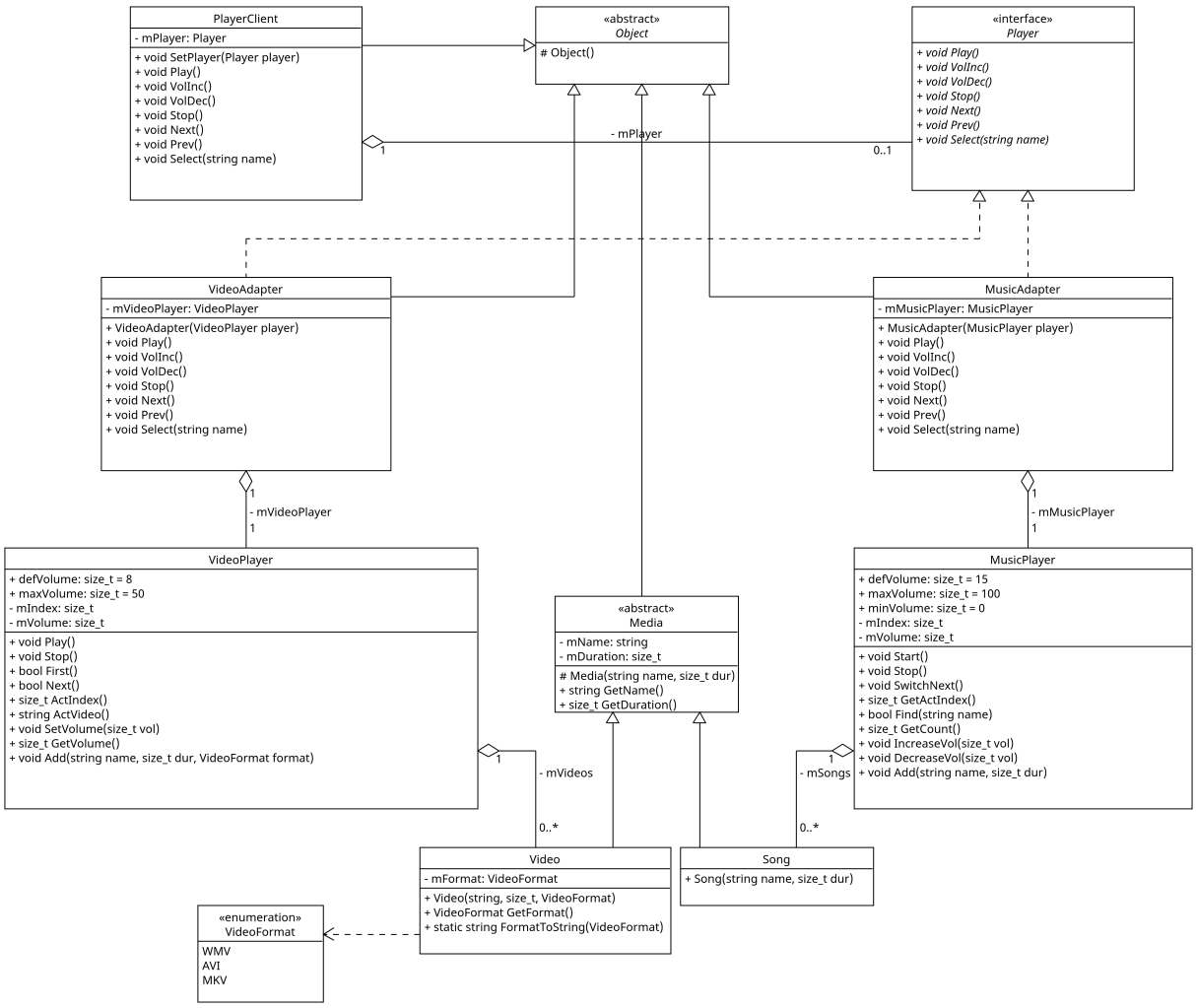


Abbildung 1: Klassendiagramm Player

Anmerkung: Das Klassendiagram steht für bessere Lesbarkeit ebenfalls im Sysdoku-Ordner zur Verfügung.

3.2 Designentscheidungen

Für die Konzeption und Implementierung der einheitlichen Player-Schnittstelle wurden gezielt objektorientierte Design-Prinzipien und Entwurfsmuster angewandt, um eine lose Kopplung und hohe Erweiterbarkeit zu garantieren.

Adapter-Pattern Da die beiden vorgegebenen Fremdklassen `VideoPlayer` (von DonkeySoft) und `MusicPlayer` (von MonkeySoft) unterschiedliche, unveränderliche Schnittstellen besitzen, wurde das **Adapter-Pattern** eingesetzt. Die Klassen `VideoAdapter` und `MusicAdapter` implementieren jeweils das einheitliche `Player`-Interface und übersetzen die Methodenaufrufe in die jeweilige proprietäre Schnittstelle. Dadurch kann der `PlayerClient` beide Medientypen über eine gemeinsame, generische Schnittstelle steuern.

Vererbungshierarchie und gemeinsame Basisklassen Zur Reduzierung von Code-Duplikaten und für ein sauberes Fundament erben alle zentralen Klassen von einer abstrakten Basisklasse `Object`. Zudem wurde für die Medienverwaltung die abstrakte Klasse `Media` eingeführt, von der die Klassen `Video` und `Song` abgeleitet sind. Gemeinsame Attribute wie Name und Dauer werden dadurch zentral in `Media` verwaltet.

Zustandsverwaltung und Kapselung Der `PlayerClient` hält eine optionale Referenz auf ein `Player`-Interface und delegiert Befehle direkt an diesen. Zur Vermeidung von Nullzeiger-Zugriffen wird im Client stets geprüft, ob ein gültiger Player gesetzt ist. Die spezifischen Restriktionen der Player (unterschiedliche Lautstärkebereiche wie 0–50 beim VideoPlayer bzw. 0–100 beim MusicPlayer) werden innerhalb der jeweiligen Adapter und Player sicher gekapselt und abgefangen.

Fehlerstrategie Fehlbedienungen (wie beispielsweise der Versuch, ein Medium bei leerer Playlist abzuspielen) werden kontrolliert abgefangen. Entsprechend den Vorgaben werden aussagekräftige Fehlermeldungen auf der Konsole ausgegeben und die betroffenen Methoden kontrolliert verlassen, ohne das Gesamtsystem zu destabilisieren.

4 Komponentenentwurf

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

5.1 Ausgabe Konsole Testtreiber

```
--- VideoPlayer ---
volume is now -> 5
Volume verified: 5
Index 0: Video1
playing video number 1: Video1 [duration -> 120 min], WMV-Format
Index 1: Video2
playing video number 2: Video2 [duration -> 90 min], AVI-Format
Index 2: Video3
playing video number 3: Video3 [duration -> 200 min], MKV-Format
stop video: Video3 [duration -> 200 min], MKV-Format
[SUCCESS] VideoPlayer passed

--- MusicPlayer ---
Count verified: 3
volume is now -> 18
volume is now -> 17
playing song number 1: Song1 (180 sec)
Index verified: 1
song: Song9 not found!
stop song: Song3 (150 sec)
[SUCCESS] MusicPlayer passed

--- VideoAdapter via Client ---
playing video number 1: Video1 [duration -> 120 min], WMV-Format
volume is now -> 9
playing video number 2: Video2 [duration -> 90 min], AVI-Format
playing video number 1: Video1 [duration -> 120 min], WMV-Format
playing video number 3: Video3 [duration -> 200 min], MKV-Format
video: NonExisting not found!
stop video: Video3 [duration -> 200 min], MKV-Format
--- MusicAdapter via Client ---
playing song number 1: Song1 (180 sec)
volume is now -> 14
playing song number 2: Song2 (210 sec)
playing song number 1: Song1 (180 sec)
playing song number 2: Song2 (210 sec)
song: NonExisting not found!
stop song: Song2 (210 sec)
[SUCCESS] Adapters passed

--- Edge Cases ---
no video in playlist!
no song in playlist!
[SUCCESS] EdgeCases passed
```

All tests successfully executed and verified!

6 Quellcode

Listing 1: ../Object.h

```
#ifndef OBJECT_H
#define OBJECT_H

/// @author Julian Rieger
/// @date 05.11.2026
/// @brief Abstract base class of all classes of the image management.
///
/// The class can not be instantiated directly (protected constructor) and it
/// prevents copying and assigning of all derived classes.
class Object
{
public:
    /// @brief Copying is not allowed.
    Object(const Object&) = delete;

    /// @brief Assigning is not allowed.
    Object& operator=(const Object&) = delete;

    /// @brief Virtual destructor, so derived objects can be deleted via a base
        pointer.
    virtual ~Object();

protected:
    /// @brief Protected constructor, only derived classes can create an Object.
    Object();
};

#endif
```

Listing 2: ../BlankCircle.h

```
#ifndef BLANKCIRCLE_H
#define BLANKCIRCLE_H
#include <cstddef>
#include <string>
#include "Circle.h"

/// @author Julian Rieger
/// @date 05.11.2026
/// @brief A not filled circle.
///
/// Objects of this class can only be created by the BlankGraphicsObjectFactory.
class BlankCircle : public Circle
{
public:
    /// @brief Creates the SVG element of this not filled circle.
    /// @returns the SVG element as text.
    std::string ToSVG() const override;

    /// @brief Creates the object.
    /// @param[in] stroke is the color of the outline.
    /// @param[in] x is the x coordinate.
    /// @param[in] y is the y coordinate.
    /// @param[in] radius is the radius of the circle.
    BlankCircle(const std::string& stroke, int x, int y, size_t radius);
};

#endif
```

Listing 3: ../BlankCircle.cpp

```
#include "BlankCircle.h"

```

```
3  std::string BlankCircle::ToSVG() const
4  {
5      //<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
6      std::string svg{};
7      svg += "<circle cx=\"";
8      svg += std::to_string(GetX());
9      svg += "\" cy=\"";
10     svg += std::to_string(GetY());
11     svg += "\" r=\"";
12     svg += std::to_string(GetRadius());
13     svg += "\" stroke=\"";
14     svg += GetStroke();
15     svg += "\" fill=\"none\"/>";
16     return svg;
17 }
18
19 BlankCircle::BlankCircle(const std::string& stroke, int x, int y, size_t radius) :
20     Circle{stroke, x, y, radius}
21 {
22 }
```

Listing 4: ../FilledCircle.h

```
1  #ifndef FILLEDCIRCLE_H
2  #define FILLEDCIRCLE_H
3  #include <cstddef>
4  #include <string>
5  #include "Circle.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief A filled circle.
10 ///
11 /// Objects of this class can only be created by the FillGraphicsObjectFactory.
12 class FilledCircle : public Circle
13 {
14 public:
15    /// @brief Creates the SVG element of this filled circle.
16    /// @returns the SVG element as text.
17    std::string ToSVG() const override;
18
19    /// @brief Creates the object.
20    /// @param[in] stroke is the color of the outline (and of the fill).
21    /// @param[in] x is the x coordinate.d
22    /// @param[in] y is the y coordinate.
23    /// @param[in] radius is the radius of the circle.
24    FilledCircle(const std::string& stroke, int x, int y, size_t radius);
25 };
26
27 #endif
```

Listing 5: ../FilledCircle.cpp

```
1  #include "FilledCircle.h"
2
3  FilledCircle::FilledCircle(const std::string& stroke, int x, int y, size_t radius)
4      : Circle(stroke, x, y, radius)
5  {
6  }
7
8  std::string FilledCircle::ToSVG() const
9  {
10     //<circle cx="55" cy="45" r="35" stroke="black"/>
11     std::string svg{};
12     svg += "<circle cx=\"";
13     svg += std::to_string(GetX());
14     svg += "\" cy=\"";
```

```
15      svg += std::to_string(GetY());
16      svg += "\" r=\"";
17      svg += std::to_string(GetRadius());
18      svg += "\" stroke=\"";
19      svg += GetStroke();
20      svg += "\"/>";
21      return svg;
22  }
```

Listing 6: ../Circle.h

```
1   #ifndef CIRCLE_H
2   #define CIRCLE_H
3   #include <cstddef>
4   #include <string>
5   #include "GraphicObject.h"
6
7   /// @author Julian Rieger
8   /// @date 05.11.2026
9   /// @brief Abstract base class of all circles.
10  ///
11  /// The position (mX, mY) is the center of the circle. Whether the circle is
12  /// filled or not is decided by the derived classes in ToSVG.
13  class Circle : public GraphicObject
14  {
15  public:
16      /// @brief Creates the SVG element of the circle, implemented by the derived
            classes.
17      /// @returns the SVG element as text.
18      std::string ToSVG() const override = 0;
19
20  protected:
21      /// @brief Creates a circle.
22      /// @param[in] stroke is the color of the outline.
23      /// @param[in] x is the x coordinate of the center.
24      /// @param[in] y is the y coordinate of the center.
25      /// @param[in] radius is the radius of the circle.
26      Circle(const std::string& stroke, int x, int y, size_t radius);
27
28      /// @brief Access function for the radius.
29      /// @returns the radius.
30      size_t GetRadius() const;
31
32  private:
33      /// @brief Radius of the circle.
34      size_t mRadius;
35  };
36
37  #endif
```

Listing 7: ../Circle.cpp

```
1   #include "Circle.h"
2
3   Circle::Circle(const std::string& stroke, int x, int y, size_t radius) :
4       GraphicObject{stroke, x, y},
5       mRadius{radius}
6   {
7   }
8
9   size_t Circle::GetRadius() const
10  {
11      return mRadius;
12  }
```

Listing 8: ../BlankRectangle.h

```h
#ifndef BLANKRECTANGLE_H
#define BLANKRECTANGLE_H
#include <cstddef>
#include <string>
#include "Rectangle.h"

/// @author Julian Rieger
/// @date 05.11.2026
/// @brief A not filled rectangle.
///
/// Objects of this class can only be created by the BlankGraphicsObjectFactory.
class BlankRectangle : public Rectangle
{
public:
    /// @brief Creates the SVG element of this not filled rectangle.
    /// @returns the SVG element as text.
    std::string ToSVG() const override;

    /// @brief Creates the object.
    /// @param[in] stroke is the color of the outline.
    /// @param[in] x is the x coordinate.
    /// @param[in] y is the y coordinate.
    /// @param[in] width is the width of the rectangle.
    /// @param[in] height is the height of the rectangle.
    BlankRectangle(const std::string& stroke, int x, int y, size_t width, size_t
        height);
};

#endif
```

Listing 9: ../BlankRectangle.cpp

```cpp
#include "BlankRectangle.h"

BlankRectangle::BlankRectangle(const std::string& stroke, int x, int y, size_t
    width, size_t height) :
    Rectangle{stroke, x, y, width, height}
{
}

std::string BlankRectangle::ToSVG() const
{
    //<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
    std::string svg{};
    svg += "<rect x=\"";
    svg += std::to_string(GetX());
    svg += "\" y=\"";
    svg += std::to_string(GetY());
    svg += "\" width=\"";
    svg += std::to_string(GetWidth());
    svg += "\" height=\"";
    svg += std::to_string(GetHeight());
    svg += "\" stroke=\"";
    svg += GetStroke();
    svg += "\" fill=\"none\"/>";
    return svg;
}
```

Listing 10: ../FilledRectangle.h

```h
#ifndef FILLEDRECTANGLE_H
#define FILLEDRECTANGLE_H
#include <cstddef>
#include <string>
#include "Rectangle.h"

```

```
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief A filled rectangle.
10 ///
11 /// Objects of this class can only be created by the FillGraphicsObjectFactory.
12 class FilledRectangle : public Rectangle
13 {
14 public:
15     /// @brief Creates the SVG element of this filled rectangle.
16     /// @returns the SVG element as text.
17     std::string ToSVG() const override;
18
19     /// @brief Creates the object.
20     /// @param[in] stroke is the color of the outline (and of the fill).
21     /// @param[in] x is the x coordinate.
22     /// @param[in] y is the y coordinate.
23     /// @param[in] width is the width of the rectangle.
24     /// @param[in] height is the height of the rectangle.
25     FilledRectangle(const std::string& stroke, int x, int y, size_t width, size_t
           height);
26 };
27
28 #endif
```

Listing 11: ../FilledRectangle.cpp

```
1  #include "FilledRectangle.h"
2
3  FilledRectangle::FilledRectangle(const std::string& stroke, int x, int y, size_t
       width, size_t height) :
4      Rectangle{stroke, x, y, width, height}
5  {
6  }
7
8  std::string FilledRectangle::ToSVG() const
9  {
10     //<rect x="10" y="10" width="90" height="70" stroke="black"/>
11     std::string svg{};
12     svg += "<rect x=\"";
13     svg += std::to_string(GetX());
14     svg += "\" y=\"";
15     svg += std::to_string(GetY());
16     svg += "\" width=\"";
17     svg += std::to_string(GetWidth());
18     svg += "\" height=\"";
19     svg += std::to_string(GetHeight());
20     svg += "\" stroke=\"";
21     svg += GetStroke();
22     svg += "\"/>";
23     return svg;
24 }
```

Listing 12: ../Rectangle.h

```
1  #ifndef RECTANGLE_H
2  #define RECTANGLE_H
3  #include <cstddef>
4  #include <string>
5  #include "GraphicObject.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief Abstract base class of all rectangles.
10 ///
11 /// The position (mX, mY) is the upper left corner of the rectangle. Whether
12 /// the rectangle is filled or not is decided by the derived classes in ToSVG.
13 class Rectangle : public GraphicObject
```

```
14 {
15 public:
16     /// @brief Creates the SVG element of the rectangle, implemented by the derived
           classes.
17     /// @returns the SVG element as text.
18     std::string ToSVG() const override = 0;
19
20 protected:
21     /// @brief Creates a rectangle.
22     /// @param[in] stroke is the color of the outline.
23     /// @param[in] x is the x coordinate of the upper left corner.
24     /// @param[in] y is the y coordinate of the upper left corner.
25     /// @param[in] width is the width of the rectangle.
26     /// @param[in] height is the height of the rectangle.
27     Rectangle(const std::string& stroke, int x, int y, size_t width, size_t height)
           ;
28
29     /// @brief Access function for the width.
30     /// @returns the width.
31     size_t GetWidth() const;
32
33     /// @brief Access function for the height.
34     /// @returns the height.
35     size_t GetHeight() const;
36
37 private:
38     /// @brief Width of the rectangle.
39     size_t mWidth;
40
41     /// @brief Height of the rectangle.
42     size_t mHeight;
43 };
44
45 #endif
```

Listing 13: ../Rectangle.cpp

```
1  #include "Rectangle.h"
2
3  Rectangle::Rectangle(const std::string& stroke, int x, int y, size_t width, size_t
       height) :
4      GraphicObject{stroke, x, y},
5      mWidth{width},
6      mHeight{height}
7  {
8  }
9
10 size_t Rectangle::GetWidth() const
11 {
12     return mWidth;
13 }
14
15 size_t Rectangle::GetHeight() const
16 {
17     return mHeight;
18 }
```

Listing 14: ../SingletonBase.h

```
1 #ifndef SINGLETONBASE_H
2 #define SINGLETONBASE_H
3
4 #include <memory>
5 #include "Object.h"
6
7 template <typename T>
8 class SingletonBase : Object
```

```
{
public:
    static T& GetInstance()
    {
        if (mInstance == nullptr)
        {
            mInstance = std::unique_ptr<T>{ new T{} };
        }

        return *mInstance;
    }

    // Free singleton before end of program
    static void Delete()
    {
        mInstance.reset();
    }

protected:
    // Protected constructor so that only derived classes can be created and
        destroyed
    SingletonBase() = default;
    virtual ~SingletonBase() = default;

private:
    SingletonBase(SingletonBase const& s) = delete;
    SingletonBase& operator=(SingletonBase const& s) = delete;

    static std::unique_ptr<T> mInstance;
};

// Initialize static member
template <typename T>
std::unique_ptr<T> SingletonBase<T>::mInstance = nullptr;

#endif
```

Listing 15: ../IGraphicsObjectFactory.h

```
#ifndef IGRAPHICSOBJECTFACTORY_H
#define IGRAPHICSOBJECTFACTORY_H
#include <cstddef>
#include <memory>
#include <string>
#include "Circle.h"
#include "Rectangle.h"

/// @author Julian Rieger
/// @date 05.11.2026
/// @brief Interface of the abstract factory which creates graphical objects.
///
/// Every concrete factory creates a family of objects (e.g. only filled or
/// only not filled ones). New families (hatched, dotted, ...) can be added by
/// implementing this interface, without changing the existing classes.
class IGraphicsObjectFactory
{
public:
   /// @brief Virtual destructor.
   virtual ~IGraphicsObjectFactory() = default;

   /// @brief Creates a circle.
   /// @param[in] stroke is the color of the outline.
   /// @param[in] x is the x coordinate of the center.
   /// @param[in] y is the y coordinate of the center.
   /// @param[in] radius is the radius of the circle.
   /// @returns the created circle, the caller owns it.
   virtual std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x,
       int y, size_t radius) const = 0;

```

```
30     /// @brief Creates a rectangle.
31     /// @param[in] stroke is the color of the outline.
32     /// @param[in] x is the x coordinate of the upper left corner.
33     /// @param[in] y is the y coordinate of the upper left corner.
34     /// @param[in] width is the width of the rectangle.
35     /// @param[in] height is the height of the rectangle.
36     /// @returns the created rectangle, the caller owns it.
37     virtual std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke,
           int x, int y, size_t width, size_t height) const = 0;
38 };
39
40 #endif
```

Listing 16: ../BlankGraphicsObjectFactory.h

```
1  #ifndef BLANKGRAPHICSOBJECTFACTORY_H
2  #define BLANKGRAPHICSOBJECTFACTORY_H
3  #include <cstddef>
4  #include <memory>
5  #include <string>
6  #include "IGraphicsObjectFactory.h"
7  #include "Object.h"
8  #include "SingletonBase.h"
9
10 /// @author Leander Kieweg
11 /// @date 05.11.2026
12 /// @brief Factory which creates not filled circles and rectangles.
13 ///
14 /// The factory exists only once (singleton). It is created at the first call
15 /// of GetInstance and released automatically when the program ends.
16 class BlankGraphicsObjectFactory : public IGraphicsObjectFactory, public
       SingletonBase<BlankGraphicsObjectFactory>
17 {
18     friend class SingletonBase<BlankGraphicsObjectFactory>;
19 public:
20     /// @brief Creates a not filled circle.
21     /// @param[in] stroke is the color of the outline.
22     /// @param[in] x is the x coordinate of the center.
23     /// @param[in] y is the y coordinate of the center.
24     /// @param[in] radius is the radius of the circle.
25     /// @returns the created circle, the caller owns it.
26     std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x, int y,
           size_t radius) const override;
27
28     /// @brief Creates a not filled rectangle.
29     /// @param[in] stroke is the color of the outline.
30     /// @param[in] x is the x coordinate of the upper left corner.
31     /// @param[in] y is the y coordinate of the upper left corner.
32     /// @param[in] width is the width of the rectangle.
33     /// @param[in] height is the height of the rectangle.
34     /// @returns the created rectangle, the caller owns it.
35     std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke, int x,
           int y, size_t width, size_t height) const override;
36
37 private:
38     /// @brief Private constructor, use GetInstance.
39     BlankGraphicsObjectFactory() = default;
40 };
41
42 #endif
```

Listing 17: ../BlankGraphicsObjectFactory.cpp

```
1 #include "BlankGraphicsObjectFactory.h"
2 #include "BlankCircle.h"
3 #include "BlankRectangle.h"
4
```

```
5  std::unique_ptr<Circle> BlankGraphicsObjectFactory::CreateCircle(const std::string
       &stroke, int x, int y,
6                                                                   size_t radius)
                                                                        const {
7      return std::make_unique<BlankCircle>(stroke, x, y, radius);
8  }
9
10 std::unique_ptr<Rectangle> BlankGraphicsObjectFactory::CreateRectangle(const std::
       string &stroke, int x, int y,
11     size_t width, size_t height) const {
12     return std::make_unique<BlankRectangle>(stroke, x, y, width, height);
13 }
```

Listing 18: ../FillGraphicsObjectFactory.h

```
1  #ifndef FILLGRAPHICSOBJECTFACTORY_H
2  #define FILLGRAPHICSOBJECTFACTORY_H
3  #include <cstddef>
4  #include <memory>
5  #include <string>
6  #include "IGraphicsObjectFactory.h"
7  #include "Object.h"
8  #include "SingletonBase.h"
9
10 /// @author Leander Kieweg
11 /// @date 05.11.2026
12 /// @brief Factory which creates filled circles and rectangles.
13 ///
14 /// The factory exists only once (singleton). It is created at the first call
15 /// of GetInstance and released automatically when the program ends.
16 class FillGraphicsObjectFactory : public IGraphicsObjectFactory, public
       SingletonBase<FillGraphicsObjectFactory>
17 {
18    friend class SingletonBase<FillGraphicsObjectFactory>;
19 public:
20
21    /// @brief Creates a filled circle.
22    /// @param[in] stroke is the color of the outline.
23    /// @param[in] x is the x coordinate of the center.
24    /// @param[in] y is the y coordinate of the center.
25    /// @param[in] radius is the radius of the circle.
26    /// @returns the created circle, the caller owns it.
27    std::unique_ptr<Circle> CreateCircle(const std::string& stroke, int x, int y,
          size_t radius) const override;
28
29    /// @brief Creates a filled rectangle.
30    /// @param[in] stroke is the color of the outline.
31    /// @param[in] x is the x coordinate of the upper left corner.
32    /// @param[in] y is the y coordinate of the upper left corner.
33    /// @param[in] width is the width of the rectangle.
34    /// @param[in] height is the height of the rectangle.
35    /// @returns the created rectangle, the caller owns it.
36    std::unique_ptr<Rectangle> CreateRectangle(const std::string& stroke, int x,
          int y, size_t width, size_t height) const override;
37
38 private:
39    /// @brief Private constructor, use GetInstance.
40    FillGraphicsObjectFactory() = default;
41 };
42
43 #endif
```

Listing 19: ../FillGraphicsObjectFactory.cpp

```
1  #include "FillGraphicsObjectFactory.h"
2  #include "FilledCircle.h"
3  #include "FilledRectangle.h"
```

```
4
5  std::unique_ptr<Circle> FillGraphicsObjectFactory::CreateCircle(const std::string
       &stroke, int x, int y,
6                                                                  size_t radius)
                                                                       const {
7      return std::make_unique<FilledCircle>(stroke, x, y, radius);
8  }
9
10 std::unique_ptr<Rectangle> FillGraphicsObjectFactory::CreateRectangle(const std::
       string &stroke, int x, int y,
11     size_t width, size_t height) const {
12     return std::make_unique<FilledRectangle>(stroke, x, y, width, height);
13 }
```

Listing 20: ../Image.h

```
1  #ifndef IMAGE_H
2  #define IMAGE_H
3  #include <memory>
4  #include <string>
5  #include <vector>
6  #include <fstream>
7  #include "GraphicObject.h"
8  #include "Object.h"
9
10 /// @author Julian Rieger
11 /// @date 05.11.2026
12 /// @brief An image which consists of graphical objects and is stored as SVG file.
13 ///
14 class Image : public Object
15 {
16 public:
17     /// @brief Creates an image and its graphical objects.
18     /// @param[in] filename is the name of the SVG file.
19     explicit Image(const std::string& filename);
20
21     /// @brief Writes the image as SVG file.
22     /// @throws std::runtime_error if the file can not be opened.
23     void WriteSVG() const;
24
25     /// @brief Adds a graphic object to the vector.
26     /// Uses push_back() internally.
27     void AddGraphicObject(std::unique_ptr<GraphicObject> obj);
28
29 private:
30
31     /// @brief Name of the SVG file.
32     std::string mFilename;
33
34     /// @brief All graphical objects of the image, owned by the image.
35     std::vector<std::unique_ptr<GraphicObject>> mGraphicObjects;
36
37     /// @brief Prints the formated header of the SVG file
38     void PrintHeader(std::ostream& ost) const;
39
40     /// @brief Prints the formated footer of the SVG file
41     void PrintFooter(std::ostream& ost) const;
42
43 };
44
45 #endif
```

Listing 21: ../Image.cpp

```
1  #include "Image.h"
2  #include <fstream>
3  #include <stdexcept>
```

```cpp
#include <utility>

Image::Image(const std::string& filename) :
    mFilename{ filename }
{
}

void Image::WriteSVG() const
{
    std::ofstream file(mFilename);
    if (!file.is_open())
        throw std::runtime_error("Error: Can not open file " + mFilename);

    PrintHeader(file);
    for (const auto& elem : mGraphicObjects)
        file << elem->ToSVG() << "\n";
    PrintFooter(file);
}

void Image::AddGraphicObject(std::unique_ptr<GraphicObject> obj)
{
    mGraphicObjects.push_back(std::move(obj));
}

void Image::PrintHeader(std::ostream& out) const
{
    out << "<?xml version=\"1.0\"?>\n"
        << "<svg xmlns=\"http://www.w3.org/2000/svg\">\n";
}

void Image::PrintFooter(std::ostream& out) const
{
    out << "</svg>\n";
}
```

Listing 22: ../GraphicsManagement.h

```cpp
#ifndef GRAPHICSMANAGEMENT_H
#define GRAPHICSMANAGEMENT_H
#include <cstddef>
#include <memory>
#include <string>
#include <vector>
#include "Circle.h"
#include "IGraphicsObjectFactory.h"
#include "Object.h"
#include "Rectangle.h"
#include "SingletonBase.h"

class Image;

/// @author Leander Kieweg
/// @date 05.11.2026
/// @brief Manages all images (singleton).
///
/// The management exists only once and is released automatically when the
/// program ends. The factory that creates the graphical objects can be
/// exchanged: each call of AddImage sets the factory for the new image.
class GraphicsManagement : public SingletonBase<GraphicsManagement>
{
   friend class SingletonBase<GraphicsManagement>;
public:
   /// @brief Creates a new image with the given factory and stores it.
   /// @param[in] fact is the factory used for all objects of the new image.
   /// @param[in] file is the name of the SVG file of the image.
   void AddImage(const IGraphicsObjectFactory& fact, const std::string& file);

   /// @brief Writes all images as SVG files.
   /// @throws std::runtime_error if a file can not be opened.
```

```
33      void WriteSVG() const;
34
35      /// @brief Creates a circle with the current factory.
36      /// @param[in] stroke is the color of the outline.
37      /// @param[in] x is the x coordinate of the center.
38      /// @param[in] y is the y coordinate of the center.
39      /// @param[in] radius is the radius of the circle.
40      /// @returns the created circle, the caller owns it.
41      /// @throws std::logic_error if no factory is set.
42      void CreateCircle(const std::string& stroke, int x, int y, size_t radius);
43
44      /// @brief Creates a rectangle with the current factory.
45      /// @param[in] stroke is the color of the outline.
46      /// @param[in] x is the x coordinate of the upper left corner.
47      /// @param[in] y is the y coordinate of the upper left corner.
48      /// @param[in] width is the width of the rectangle.
49      /// @param[in] height is the height of the rectangle.
50      /// @returns the created rectangle, the caller owns it.
51      /// @throws std::logic_error if no factory is set.
52      void CreateRectangle(const std::string& stroke, int x, int y, size_t width,
            size_t height) const;
53
54  private:
55      /// @brief Private constructor, use GetInstance.
56      GraphicsManagement() = default;
57
58      /// @brief Private destructor, use Delete.
59      ~GraphicsManagement() override = default;
60
61      /// @brief Factory for the next images, not owned (the factories are singletons
            ).
62      const IGraphicsObjectFactory* mFactory;
63
64      /// @brief All images, owned by the management.
65      std::vector<std::unique_ptr<Image>> mImages;
66  };
67
68  #endif
```

Listing 23: ../GraphicsManagement.cpp

```
1   #include "GraphicsManagement.h"
2   #include "Image.h"
3   #include <stdexcept>
4
5
6   void GraphicsManagement::AddImage(const IGraphicsObjectFactory &fact, const std::
        string &file) {
7       mFactory = &fact;
8       mImages.push_back(std::make_unique<Image>(file));
9   }
10
11  void GraphicsManagement::WriteSVG() const {
12      for (const auto &image : mImages) {
13          image->WriteSVG();
14      }
15  }
16
17  void GraphicsManagement::CreateCircle(const std::string &stroke, int x, int y,
        size_t radius) {
18      if (mFactory == nullptr) {
19          throw std::logic_error("No factory is set.");
20      }
21
22      if (mImages.empty()) {
23          throw std::logic_error("No image has been added.");
24      }
25
26      mImages.back()->AddGraphicObject(
```

```
27          mFactory->CreateCircle(stroke, x, y, radius));
28  }
29
30  void GraphicsManagement::CreateRectangle(const std::string &stroke, int x, int y,
        size_t width,
31                                           size_t height) const {
32      if (mFactory == nullptr) {
33          throw std::logic_error("No factory is set.");
34      }
35
36      if (mImages.empty()) {
37          throw std::logic_error("No image has been added.");
38      }
39
40      mImages.back()->AddGraphicObject(
41          mFactory->CreateRectangle(stroke, x, y, width, height));
42  }
```

Listing 24: ../main.cpp

```
1  #include <exception>
2  #include <iostream>
3  #include "BlankGraphicsObjectFactory.h"
4  #include "FillGraphicsObjectFactory.h"
5  #include "GraphicsManagement.h"
6
7  /// @author Julian Rieger
8  /// @date 05.11.2026
9  /// @brief Test driver: creates one image with not filled and one with filled
       objects.
10 int main()
11 {
12     std::cout << "Hello ";
13     return 0;
14 }
```